



Programación Distribuida con Java-RMI



RMI (Remote Method Invocation) es el mecanismo de Java para invocar métodos de **objetos remotos**, es decir, objetos que **no se están ejecutando en la misma Máquina Virtual Java (JVM)**, normalmente en otra máquina conectada por red.

Explicación

RMI se apoya en protocolos de red para permitir la comunicación entre máquinas distintas.

A bajo nivel, la comunicación se realiza típicamente sobre **TCP**, utilizando puertos para identificar los procesos que se ejecutan en cada máquina. Sin embargo, RMI opera a un **nivel de abstracción mucho más alto**, ocultando por completo estos detalles al programador.

Conceptualmente, RMI está relacionado con el modelo **RPC (Remote Procedure Call)**, ya que permite invocar operaciones remotas con una sintaxis similar a una llamada local. No obstante, RMI es una **extensión orientada a objetos del modelo RPC**:

- En RPC se invocan **procedimientos o funciones remotas**.
- En RMI se invocan **métodos de objetos remotos**.

Para conseguir esta transparencia, RMI utiliza **resguardos (stubs)** que actúan como **proxies** del objeto remoto.

- El stub intercepta la llamada del cliente.
- Serializa los parámetros.

- Envía la petición por red.
- Bloquea al cliente hasta recibir la respuesta.
- Deserializa el valor de retorno o la excepción.

Desde el punto de vista del programador, la llamada parece local, aunque en realidad se ejecuta de forma remota.

RMI en Java

RMI permite construir **objetos distribuidos** en Java.

Estos objetos:

- se ejecutan en una JVM distinta (posiblemente en otra máquina),
- son accedidos remotamente mediante interfaces remotas,
- intercambian datos mediante **serialización de objetos**.

Al tratarse de un sistema distribuido, la arquitectura clásica es **cliente-servidor**:

- **Servidor**: publica uno o más objetos remotos.
- **Cliente**: obtiene una referencia remota a esos objetos e invoca sus métodos.

Servidor

El servidor es el **objeto que proporciona los servicios remotos**.

Características clave:

- Implementa una **interfaz remota** (`extends Remote`).
- Sus métodos deben declarar `throws RemoteException` .
- Normalmente extiende `UnicastRemoteObject` (o es exportado explícitamente).

El servidor registra el objeto remoto en el **RMI Registry**, para que los clientes puedan localizarlo.

Nota: no es obligatorio extender directamente `RemoteObject`; lo habitual y esperado en la práctica es `UnicastRemoteObject`.

Cliente

El cliente contiene referencias a **objetos remotos**, no a implementaciones locales.

- Estas referencias son stubs obtenidos desde el **registro RMI**.
- El cliente invoca métodos sobre interfaces remotas.
- Cada llamada puede implicar comunicación por red y, por tanto, es potencialmente costosa y bloqueante.

El cliente es el **consumidor de servicios**.

¿Cómo se pasan los parámetros en RMI?

RMI define reglas claras para el paso de parámetros y valores de retorno:

- **Tipos primitivos** → se pasan **por valor**.
- **Objetos serializables** (`implements Serializable`) → se pasan **por valor** (copia serializada).
- **Objetos remotos** (`extends Remote`) → se pasan **por referencia remota** (en realidad se pasa un *stub*).

Esto evita ambigüedades y permite distinguir claramente entre copia de datos y acceso remoto.

Fases de diseño RMI

El desarrollo de una aplicación RMI sigue una secuencia bien definida:

1. Interfaz remota

Define el contrato del servicio (`extends Remote`).

2. Servidor

Implementa la interfaz remota y exporta el objeto.

3. Stubs

Actúan como proxies del objeto remoto.

- Antes se generaban manualmente (`rmic`).
- Hoy se generan **dinámicamente** en tiempo de ejecución.

4. Registro

El servidor publica el objeto remoto en el **RMI Registry**.

5. Cliente

Localiza el objeto remoto y realiza las invocaciones.